walmart

September 3, 2024

1 About Walmart

Walmart is an American multinational retail corporation that operates a chain of supercenters, discount departmental stores, and grocery stores from the United States. Walmart has more than 100 million customers worldwide.

2 Objective

To analyze and compare the spending behavior of male and female customers at Walmart, particularly focusing on whether women spend more than men on Black Friday. This analysis will help Walmart's management make data-driven decisions regarding targeted marketing strategies and promotions, considering the potential differences in purchasing patterns across genders. The goal is to determine if significant differences exist in the average purchase amounts between the 50 million male and 50 million female customers.

3 Features of the Dataset

User_ID: User ID

Product_ID: Product ID

Gender: Sex of User

Age: Age in bins

Occupation: Occupation(Masked)

City_Category: Category of the City (A,B,C)

StayInCurrentCityYears: Number of years stay in current city

Marital_Status: Marital Status

ProductCategory: Product Category (Masked)

Purchase: Purchase Amount

[103]:
```
#Importing packages
import numpy as np
import pandas as pd

# Importing matplotlib and seaborn for graphs
```

```
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns
sns.set(style='whitegrid')

import warnings
warnings.filterwarnings('ignore')

from scipy import stats
from scipy.stats import kstest
import statsmodels.api as sm

# Importing Date & Time util modules
from dateutil.parser import parse

import statistics
from scipy.stats import norm
from scipy.stats import ttest_ind

import seaborn as sns
```

[73]:
```
df = pd.read_csv("Walmart_Case_Study.csv")

# Display the first few rows of the dataset
print(df.head())

# Display the last few rows of the dataset
print(df.tail())
```

```
   User_ID Product_ID Gender   Age  Occupation City_Category  \
0  1000001  P00069042      F  0-17          10             A
1  1000001  P00248942      F  0-17          10             A
2  1000001  P00087842      F  0-17          10             A
3  1000001  P00085442      F  0-17          10             A
4  1000002  P00285442      M   55+          16             C

  Stay_In_Current_City_Years  Marital_Status  Product_Category  Purchase
0                          2               0                 3      8370
1                          2               0                 1     15200
2                          2               0                12      1422
3                          2               0                12      1057
4                         4+               0                 8      7969
        User_ID Product_ID Gender    Age  Occupation City_Category  \
550063  1006033  P00372445      M  51-55          13             B
550064  1006035  P00375436      F  26-35           1             C
550065  1006036  P00375436      F  26-35          15             B
550066  1006038  P00375436      F    55+           1             C
```

```
550067  1006039  P00371644      F  46-50           0              B

       Stay_In_Current_City_Years  Marital_Status  Product_Category  Purchase
550063                          1               1                20       368
550064                          3               0                20       371
550065                         4+               1                20       137
550066                          2               0                20       365
550067                         4+               1                20       490
```

Checking the statistics and characterstics of the data set

[74]:
```
# Display basic information about the dataset
print(df.info())
print()

print(df.shape)
print()

# Display summary statistics for numerical columns
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
 #   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
 0   User_ID                     550068 non-null  int64
 1   Product_ID                  550068 non-null  object
 2   Gender                      550068 non-null  object
 3   Age                         550068 non-null  object
 4   Occupation                  550068 non-null  int64
 5   City_Category               550068 non-null  object
 6   Stay_In_Current_City_Years  550068 non-null  object
 7   Marital_Status              550068 non-null  int64
 8   Product_Category            550068 non-null  int64
 9   Purchase                    550068 non-null  int64
dtypes: int64(5), object(5)
memory usage: 42.0+ MB
None

(550068, 10)

            User_ID     Occupation  Marital_Status  Product_Category  \
count  5.500680e+05  550068.000000   550068.000000     550068.000000
mean   1.003029e+06       8.076707        0.409653          5.404270
std    1.727592e+03       6.522660        0.491770          3.936211
min    1.000001e+06       0.000000        0.000000          1.000000
25%    1.001516e+06       2.000000        0.000000          1.000000
```

```
50%    1.003077e+06       7.000000         0.000000          5.000000
75%    1.004478e+06      14.000000         1.000000          8.000000
max    1.006040e+06      20.000000         1.000000         20.000000
```

```
            Purchase
count  550068.000000
mean     9263.968713
std      5023.065394
min        12.000000
25%      5823.000000
50%      8047.000000
75%     12054.000000
max     23961.000000
```

Insights

From the above analysis, it is clear that, data has total of 10 features with lots of mixed alpha numeric data. Occupation, Maritial Status, Product Category and Purchase Column are integer columns; and rest all the other data types are of categorical type. We will change the datatypes of all such columns to category

Detecting Null Values

[75]:
```
# Step 2: Missing Value
print(df.isnull().sum())
```

```
User_ID                     0
Product_ID                  0
Gender                      0
Age                         0
Occupation                  0
City_Category               0
Stay_In_Current_City_Years  0
Marital_Status              0
Product_Category            0
Purchase                    0
dtype: int64
```

[76]:
```
def missingValue(df):
    #Identifying Missing data. Already verified above. To be sure again␣
 ↪checking.
    total_null = df.isnull().sum().sort_values(ascending = False)
    percent = ((df.isnull().sum()/df.isnull().count())*100).
 ↪sort_values(ascending = False)
    print("Total records = ", df.shape[0])

    md = pd.concat([total_null,percent.round(2)],axis=1,keys=['Total␣
 ↪Missing','In Percent'])
    return md
```

```
missingValue(df)
```

```
Total records =  550068
```

[76]:

	Total Missing	In Percent
User_ID	0	0.0
Product_ID	0	0.0
Gender	0	0.0
Age	0	0.0
Occupation	0	0.0
City_Category	0	0.0
Stay_In_Current_City_Years	0	0.0
Marital_Status	0	0.0
Product_Category	0	0.0
Purchase	0	0.0

Converting integer value to Category type

[77]:
```
df['Marital_Status'] = df['Marital_Status'].map({0: 'Unmarried', 1: 'Married'})
df['Product_Category'] = df['Product_Category'].astype('category')
df['Occupation'] = df['Occupation'].astype('category')

df.dtypes
```

[77]:
```
User_ID                        int64
Product_ID                    object
Gender                        object
Age                           object
Occupation                  category
City_Category                 object
Stay_In_Current_City_Years    object
Marital_Status                object
Product_Category            category
Purchase                       int64
dtype: object
```

#Duplicate Detection

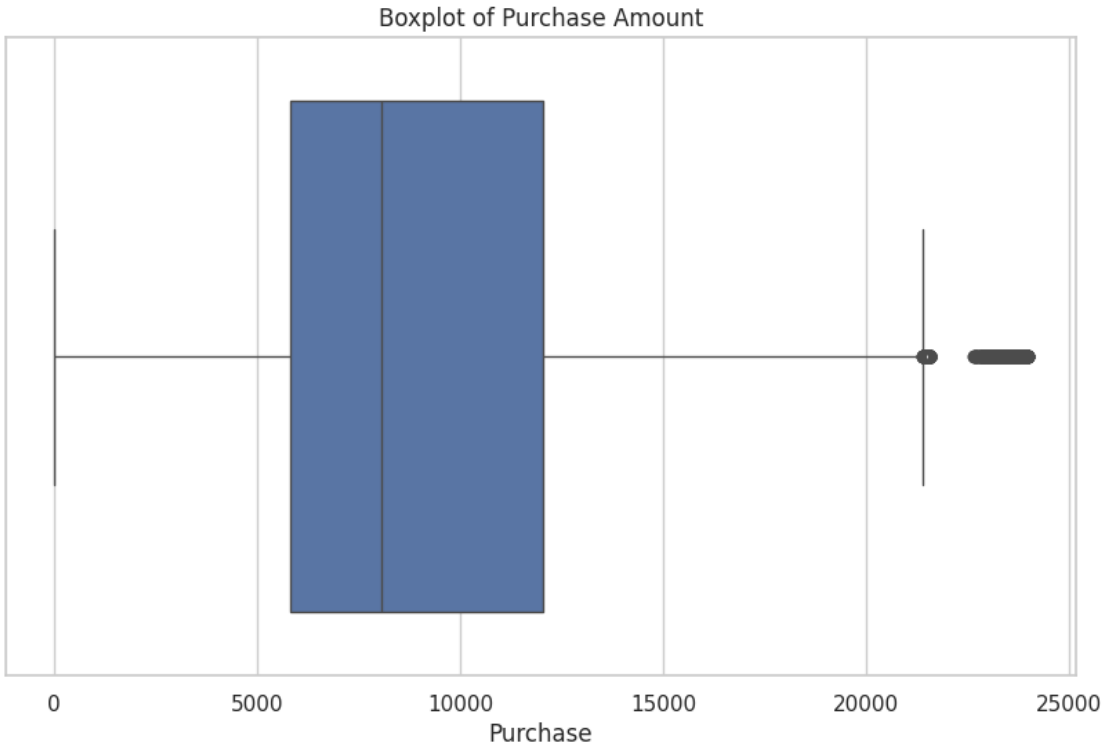
[78]:
```
df.duplicated().value_counts()
```

[78]:
```
False    550068
Name: count, dtype: int64
```

Insights There are no duplicate entries in the dataset

Detecting Outliers

[79]:
```
# Visualize outliers in the 'Purchase' column using a boxplot
plt.figure(figsize=(10, 6))
sns.boxplot(x=df['Purchase'])
plt.title('Boxplot of Purchase Amount')
plt.show()
```



Changing the Datatype of Columns

[80]:
```
for i in df.columns[:-1]:
 df[i] = df[i].astype('category')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
 #   Column                      Non-Null Count   Dtype
---  ------                      --------------   -----
 0   User_ID                     550068 non-null  category
 1   Product_ID                  550068 non-null  category
 2   Gender                      550068 non-null  category
 3   Age                         550068 non-null  category
 4   Occupation                  550068 non-null  category
 5   City_Category               550068 non-null  category
```

```
 6   Stay_In_Current_City_Years  550068 non-null  category
 7   Marital_Status              550068 non-null  category
 8   Product_Category            550068 non-null  category
 9   Purchase                    550068 non-null  int64
dtypes: category(9), int64(1)
memory usage: 10.3 MB
```

Statistical Summary

[81]:
```
# Check for categorical columns
categorical_cols = df.select_dtypes(include='category').columns

if categorical_cols.empty:
    print("No categorical columns found in the DataFrame.")
else:
    # Statistical Analysis for categorical columns
    print(df.describe(include='category'))
```

```
        User_ID Product_ID  Gender     Age  Occupation City_Category  \
count    550068     550068  550068  550068      550068        550068
unique     5891       3631       2       7          21             3
top     1001680  P00265242       M   26-35           4             B
freq       1026       1880  414259  219587       72308        231173

       Stay_In_Current_City_Years Marital_Status  Product_Category
count                      550068         550068            550068
unique                          5              2                20
top                             1      Unmarried                 5
freq                       193821         324731            150933
```

Satistical summary of numerical data type columns

[82]:
```
df.describe()
```

[82]:
```
            Purchase
count  550068.000000
mean     9263.968713
std      5023.065394
min        12.000000
25%      5823.000000
50%      8047.000000
75%     12054.000000
max     23961.000000
```

Insights

The purchase amounts vary widely, with the minimum recorded purchase being 12 and the maximum reaching 23961 . The median purchase amount of 8047 is notably lower than the mean purchase amount of 9264 , indicating a right-skewed distribution where a few high-value purchases pull up the mean

DATA EXPLORATION

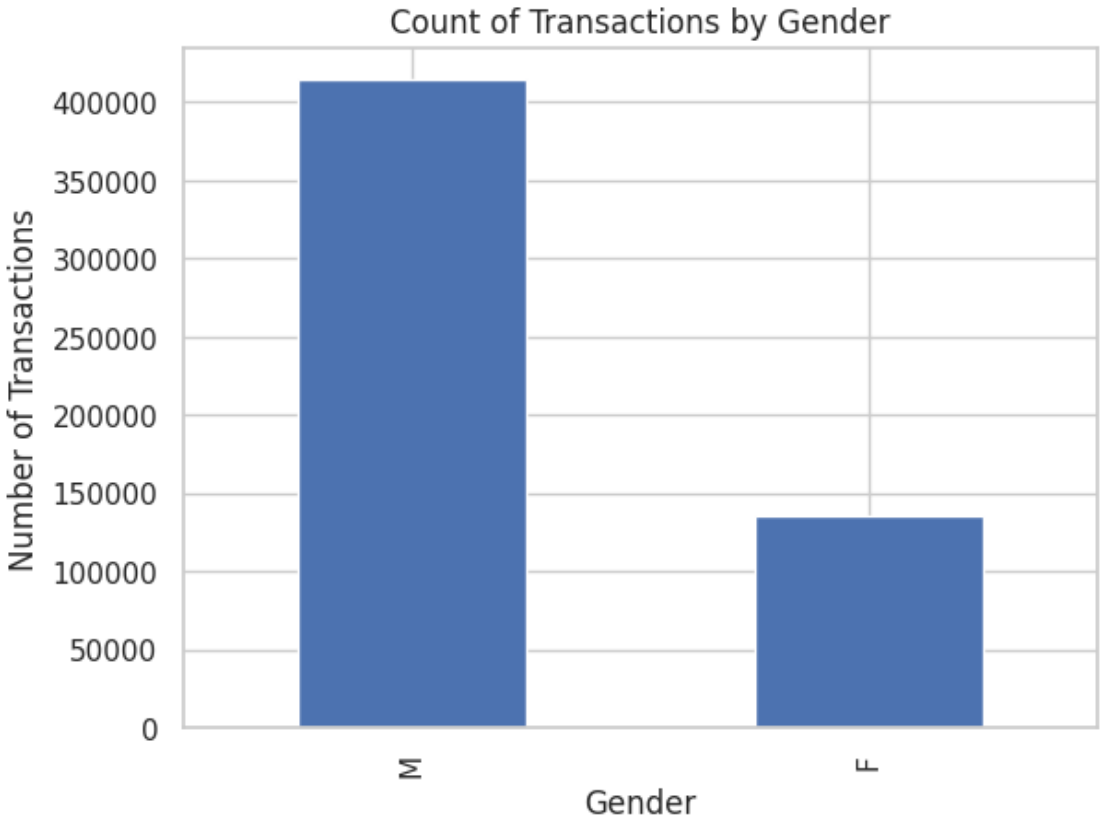
Tracking the amount spent per transaction

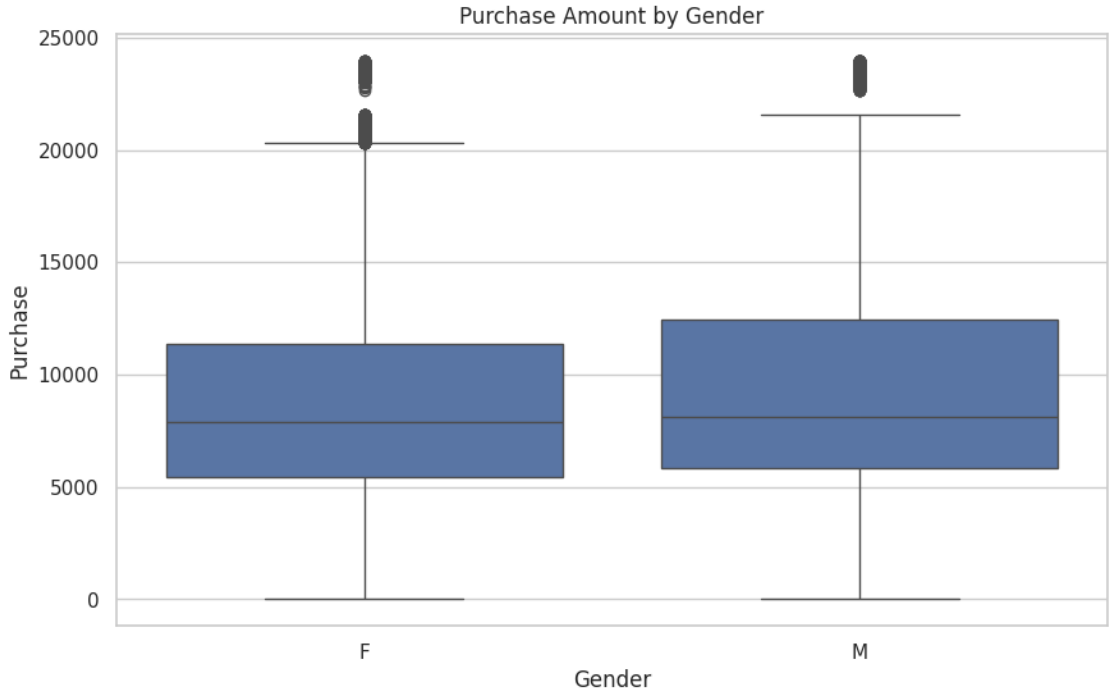
[83]:
```
# Filter data for female and male customers
female_customers = df[df['Gender'] == 'F']
male_customers = df[df['Gender'] == 'M']

# Display the number of records for each gender to confirm the size
print(f"Number of female customers: {female_customers.shape[0]}")
print(f"Number of male customers: {male_customers.shape[0]}")

# Calculate average purchase amount for female customers
avg_purchase_female = female_customers['Purchase'].mean()
print(f"Average purchase amount for female customers: ${avg_purchase_female:.
 ↪2f}")

# Calculate average purchase amount for male customers
avg_purchase_male = male_customers['Purchase'].mean()
print(f"Average purchase amount for male customers: ${avg_purchase_male:.2f}")
```

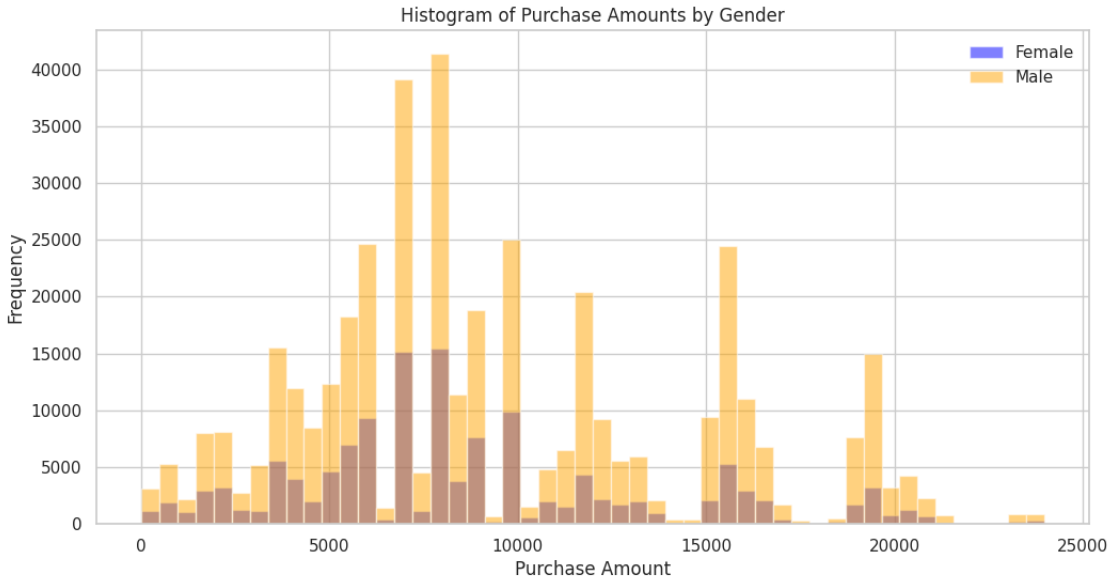
```
Number of female customers: 135809
Number of male customers: 414259
Average purchase amount for female customers: $8734.57
Average purchase amount for male customers: $9437.53
```

[84]:
```
# Combine gender data for visualization
df['Gender'].value_counts().plot(kind='bar', title='Count of Transactions by␣
 ↪Gender')
plt.ylabel('Number of Transactions')
plt.show()

# Boxplot for purchase amount by gender
plt.figure(figsize=(10, 6))
sns.boxplot(x='Gender', y='Purchase', data=df)
plt.title('Purchase Amount by Gender')
plt.show()

# Histogram of purchase amounts by gender
plt.figure(figsize=(12, 6))
plt.hist(female_customers['Purchase'], bins=50, alpha=0.5, label='Female',␣
 ↪color='blue')
plt.hist(male_customers['Purchase'], bins=50, alpha=0.5, label='Male',␣
 ↪color='orange')
plt.title('Histogram of Purchase Amounts by Gender')
plt.xlabel('Purchase Amount')
plt.ylabel('Frequency')
plt.legend(loc='upper right')
```

```
plt.show()
```


Count of Transactions by Gender
Number of Transactions
400000
350000
300000
250000
200000
150000
100000
50000
0
M
F
Gender

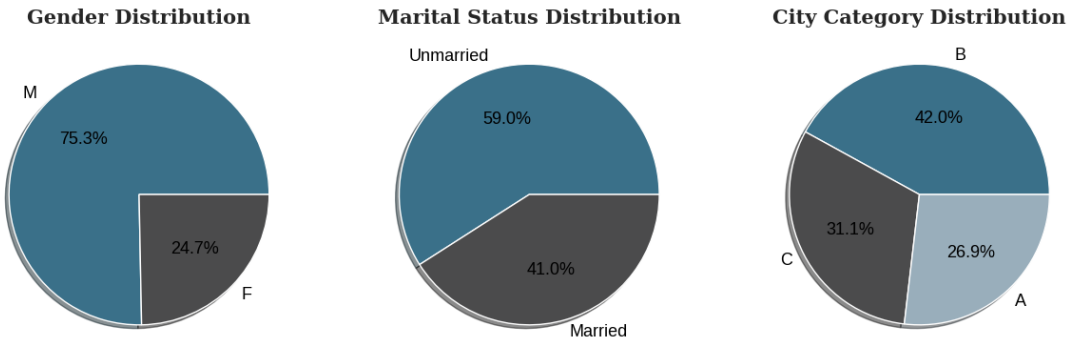




Gender, Marital Status and City Category Distribution

[85]:
```
# Set the plot style
plt.style.use('seaborn-darkgrid')
fig, axes = plt.subplots(1, 3, figsize=(15, 12))
```

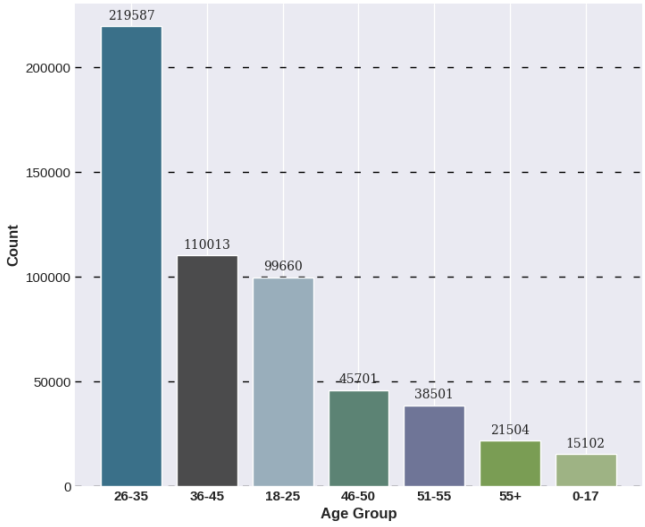
```
# Define color maps
color_map_gender = ["#3A7089", "#4b4b4c"]
color_map_marital_status = ["#3A7089", "#4b4b4c"]
color_map_city_category = ["#3A7089", "#4b4b4c", '#99AEBB']

# Create pie charts
axes[0].pie(df['Gender'].value_counts(), labels=df['Gender'].value_counts().
 ↪index, autopct='%.1f%%',
           shadow=True, colors=color_map_gender, textprops={'fontsize': 13,␣
 ↪'color': 'black'})
axes[0].set_title('Gender Distribution', fontdict={'family': 'serif', 'size':␣
 ↪15, 'weight': 'bold'})

axes[1].pie(df['Marital_Status'].value_counts(), labels=df['Marital_Status'].
 ↪value_counts().index, autopct='%.1f%%',
           shadow=True, colors=color_map_marital_status, textprops={'fontsize':
 ↪ 13, 'color': 'black'})
axes[1].set_title('Marital Status Distribution', fontdict={'family': 'serif',␣
 ↪'size': 15, 'weight': 'bold'})

axes[2].pie(df['City_Category'].value_counts(), labels=df['City_Category'].
 ↪value_counts().index, autopct='%.1f%%',
           shadow=True, colors=color_map_city_category, textprops={'fontsize':␣
 ↪13, 'color': 'black'})
axes[2].set_title('City Category Distribution', fontdict={'family': 'serif',␣
 ↪'size': 15, 'weight': 'bold'})

# Show the plots
plt.show()
```



Customer Age Distribution

[86]:
```
import matplotlib.pyplot as plt

# Set the plot style and figure layout
fig, axes = plt.subplots(1, 2, figsize=(15, 7), gridspec_kw={'width_ratios': [0.
 ↪6, 0.4]})

# Color maps
color_map_age = ["#3A7089", "#4b4b4c", '#99AEBB', '#5C8374', '#6F7597',␣
 ↪'#7A9D54', '#9EB384']
color_map_table = [["#3A7089", '#FFFFFF'], ["#4b4b4c", '#FFFFFF'], ['#99AEBB',␣
 ↪'#FFFFFF'],
                   ['#5C8374', '#FFFFFF'], ['#6F7597', '#FFFFFF'], ['#7A9D54',␣
 ↪'#FFFFFF'],
                   ['#9EB384', '#FFFFFF']]

# Bar chart for Age Distribution
temp = df['Age'].value_counts()
axes[0].bar(temp.index, temp.values, color=color_map_age, zorder=2)

# Add value counts to bars
for i in temp.index:
    axes[0].text(i, temp[i] + 5000, temp[i], ha='center', va='center',␣
 ↪fontdict={'font': 'serif', 'size': 10})

# Add grid lines and customize axes
axes[0].grid(color='black', linestyle='--', axis='y', zorder=0, dashes=(5, 10))
axes[0].spines[['top', 'left', 'right']].set_visible(False)
axes[0].set_ylabel('Count', fontweight='bold', fontsize=12)
axes[0].set_xlabel('Age Group', fontweight='bold', fontsize=12)
axes[0].set_xticklabels(temp.index, fontweight='bold')

# Info table for Age
age_info = [['26-35', '40%'], ['36-45', '20%'], ['18-25', '18%'],
            ['46-50', '8%'], ['51-55', '7%'], ['55+', '4%'], ['0-17', '3%']]
table = axes[1].table(cellText=age_info, cellColours=color_map_table,␣
 ↪cellLoc='center',
                      colLabels=['Age Group', 'Percent Dist.'],␣
 ↪colLoc='center', bbox=[0, 0, 1, 1])
table.set_fontsize(15)

# Remove axis and add title
axes[1].axis('off')
fig.suptitle('Customer Age Distribution', font='serif', size=18, weight='bold')

# Show the plot
plt.show()
```

Customer Age Distribution



Age Group	Percent Dist.
26-35	40%
36-45	20%
18-25	18%
46-50	8%
51-55	7%
55+	4%
0-17	3%

Insights

The age group of 26-35 represents the largest share of Walmart's Black Friday sales, accounting for 40% of the sales. This suggests that the young and middle-agedadults are the most active and interested in shopping for deals and discounts.

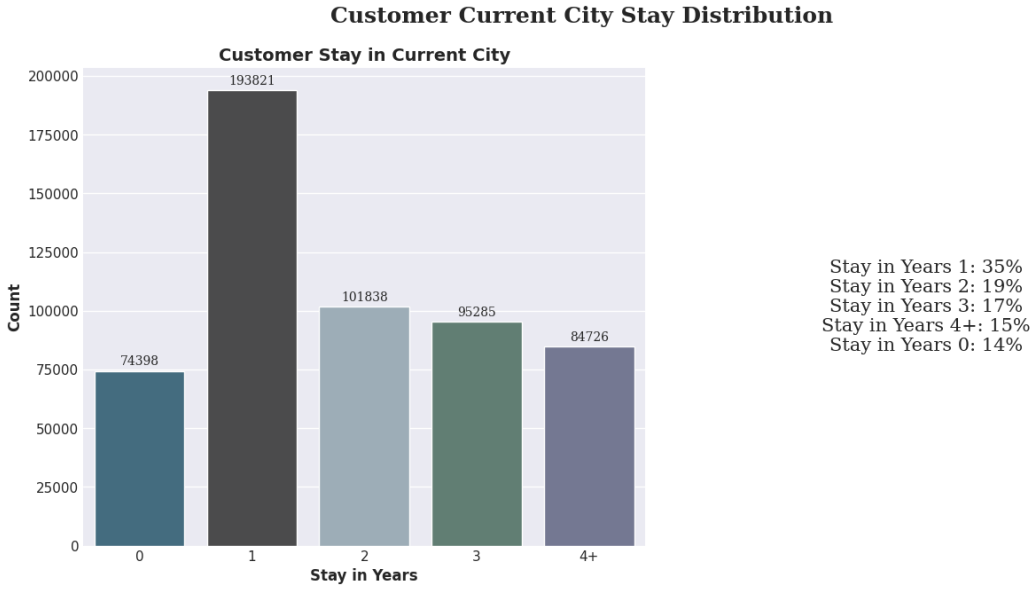
The 36-45 and 18-25 age groups are the second and third largest segments, respectively, with 20% and 18% of the sales. This indicates that Walmart has a diverse customer base that covers different life stages and preferences.

The 46-50, 51-55, 55+, and 0-17 age groups are the smallest customer segments , with less than 10% of the total sales each. This implies that Walmart may need to improve its marketing strategies and product offerings to attract more customers from these age groups, especially the seniors and the children.

Customer Stay In current City Distribution

[87]:
```
# Set the plot style and figure layout
fig, axes = plt.subplots(1, 2, figsize=(15, 7), gridspec_kw={'width_ratios': [0.
↪6, 0.4]})
sns.set_style("whitegrid")

# Bar chart for Customer Stay in Current City
temp = df['Stay_In_Current_City_Years'].value_counts().sort_index()
color_map = ["#3A7089", "#4b4b4c", '#99AEBB', '#5C8374', '#6F7597']

sns.barplot(x=temp.index, y=temp.values, palette=color_map, ax=axes[0])
axes[0].set_ylabel('Count', fontweight='bold', fontsize=12)
axes[0].set_xlabel('Stay in Years', fontweight='bold', fontsize=12)
```

```
axes[0].set_title('Customer Stay in Current City', fontweight='bold',␣
 ↪fontsize=14)

# Adding value counts on top of the bars
for i, value in enumerate(temp.values):
    axes[0].text(i, value + 4000, str(value), ha='center', va='center',␣
 ↪fontdict={'font': 'serif', 'size': 10})

# Info as text instead of a table
stay_info = {'1': '35%', '2': '19%', '3': '17%', '4+': '15%', '0': '14%'}
info_text = '\n'.join([f'Stay in Years {k}: {v}' for k, v in stay_info.items()])

# Displaying the info text
axes[1].text(0.5, 0.5, info_text, ha='center', va='center', fontdict={'font':␣
 ↪'serif', 'size': 15})
axes[1].axis('off')

# Setting the title for the figure
fig.suptitle('Customer Current City Stay Distribution', font='serif', size=18,␣
 ↪weight='bold')
plt.show()
```



Insights

The data suggests that the customers are either new to the city or move frequently, and may have different preferences and needs than long-term residents.

The majority of the customers (49%) have stayed in the current city for one year or less . This

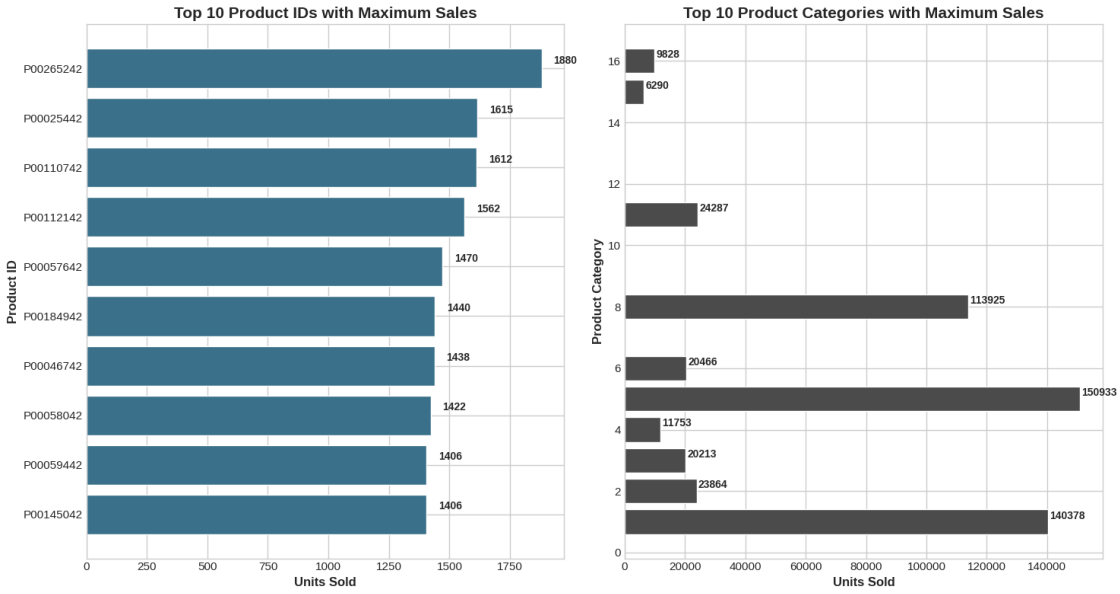
suggests that Walmart has a strong appeal to newcomers who maybe looking for affordable and convenient shopping options.

4+ years category (14%) customers indicates that Walmart has a loyal customer base who have been living in the same city for a long time.

The percentage of customers decreases as the stay in the current city increases which suggests that Walmart may benefit from targeting long-term residents for loyalty programs and promotions .

Top 10 Products and Categories: Sales Snapshot

[101]:
```
# Set the plot style
plt.style.use('seaborn-whitegrid')

# Create the figure and axes
fig, axes = plt.subplots(1, 2, figsize=(15, 8))

# Top 10 Product_ID Sales
top_products = df['Product_ID'].value_counts().head(10).sort_values()
axes[0].barh(top_products.index, top_products.values, color='#3A7089')

# Customize plot
axes[0].set_title('Top 10 Product IDs with Maximum Sales', fontweight='bold',␣
 ↪fontsize=15)
axes[0].set_xlabel('Units Sold', fontweight='bold', fontsize=12)
axes[0].set_ylabel('Product ID', fontweight='bold', fontsize=12)
for i in axes[0].patches:
    axes[0].text(i.get_width() + 50, i.get_y() + 0.5, str(i.get_width()),␣
 ↪fontsize=10, fontweight='bold')

# Top 10 Product Category Sales
top_categories = df['Product_Category'].value_counts().head(10).sort_values()
axes[1].barh(top_categories.index, top_categories.values, color='#4b4b4c')

# Customize plot
axes[1].set_title('Top 10 Product Categories with Maximum Sales',␣
 ↪fontweight='bold', fontsize=15)
axes[1].set_xlabel('Units Sold', fontweight='bold', fontsize=12)
axes[1].set_ylabel('Product Category', fontweight='bold', fontsize=12)
for i in axes[1].patches:
    axes[1].text(i.get_width() + 500, i.get_y() + 0.5, str(i.get_width()),␣
 ↪fontsize=10, fontweight='bold')

# Use tight layout to ensure everything fits well
plt.tight_layout()
plt.show()
```

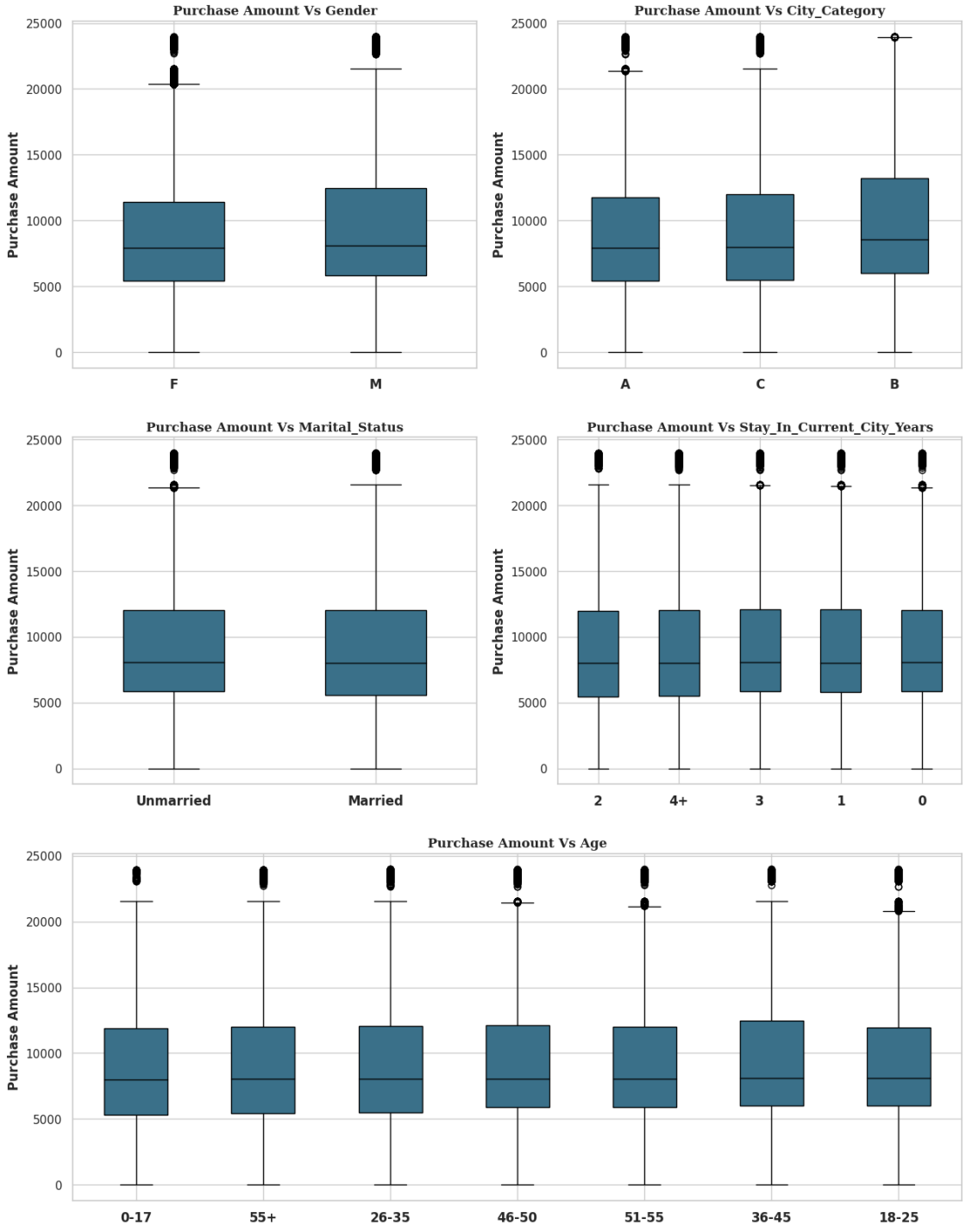


4 Bivariate Analysis

Exploring Purchase Patterns

[106]:
```
import matplotlib.pyplot as plt
import numpy as np

# Create a figure with a GridSpec layout
fig = plt.figure(figsize=(15, 20))
gs = fig.add_gridspec(3, 2)

# Define the columns and their titles
columns = [(0, 0, 'Gender'), (0, 1, 'City_Category'), (1, 0, 'Marital_Status'),
           (1, 1, 'Stay_In_Current_City_Years'), (2, 1, 'Age')]

# Define color map
color_map = ["#3A7089", "#4b4b4c", '#99AEBB', '#5C8374', '#6F7597', '#7A9D54',␣
 ↪'#9EB384']

for i, j, k in columns:
    # Plot position
    if i <= 1:
        ax0 = fig.add_subplot(gs[i, j])
    else:
        ax0 = fig.add_subplot(gs[i, :])

    # Data preparation for boxplot
```

```
    grouped = df.groupby(k)['Purchase'].apply(list)

    # Create box plots
    positions = np.arange(len(grouped))
    ax0.boxplot([grouped[x] for x in positions], positions=positions, widths=0.
 ↪5, patch_artist=True,
                boxprops=dict(facecolor=color_map[0]),
                medianprops=dict(color='black'))

    # Customizing axis
    ax0.set_title(f'Purchase Amount Vs {k}', fontsize=12, weight='bold',␣
 ↪family='serif')
    ax0.set_xticks(positions)
    ax0.set_xticklabels(df[k].unique(), fontweight='bold', fontsize=12)
    ax0.set_ylabel('Purchase Amount', fontweight='bold', fontsize=12)
    ax0.set_xlabel('')

plt.show()
```



Statistical Significance

[88]:
```
# Confidence Interval Calculation
sample_mean = df.groupby('Gender')['Purchase'].mean()
sample_std = df.groupby('Gender')['Purchase'].std()
```

```
n = len(df['User_ID'].unique())

confidence_level = 0.95
z = stats.norm.ppf((1 + confidence_level) / 2)
margin_of_error = z * (sample_std / np.sqrt(n))
confidence_interval = (sample_mean - margin_of_error, sample_mean +␣
 ↪margin_of_error)
print(confidence_interval)
```

```
(Gender
F    8612.829497
M    9307.491761
Name: Purchase, dtype: float64, Gender
F    8856.302033
M    9567.560320
Name: Purchase, dtype: float64)
```

Male customers spend significantly more per transaction than female customers, as evidenced by non-overlapping confidence intervals. Walmart should target male customers with premium product offerings and bundles, while encouraging higher spending among female customers through tailored promotions and loyalty programs.

[89]:
```
# Perform t-test for independent samples
t_stat, p_value = ttest_ind(female_customers['Purchase'],␣
 ↪male_customers['Purchase'])
print(f"T-statistic: {t_stat}, P-value: {p_value}")
```

```
T-statistic: -44.837957934353966, P-value: 0.0
```

Interpretation As the p-value < 0.05: The difference in average spending is statistically significant.

Calculate the Sample Mean and Standard Deviation For female

[90]:
```
# Sample mean and standard deviation for female customers
mean_female = female_customers['Purchase'].mean()
std_dev_female = female_customers['Purchase'].std()
n_female = female_customers.shape[0]

print(f"Sample Mean (Female): ${mean_female:.2f}")
print(f"Sample Standard Deviation (Female): ${std_dev_female:.2f}")
print(f"Sample Size (Female): {n_female}")
```

```
Sample Mean (Female): $8734.57
Sample Standard Deviation (Female): $4767.23
Sample Size (Female): 135809
```

[91]:
```
# Z score at 95% confidence level For female customer
```

```
# Confidence level
confidence = 0.95
z_score = stats.norm.ppf((1 + confidence) / 2)

# Margin of error
margin_of_error_female = z_score * (std_dev_female / np.sqrt(n_female))

# Confidence Interval
ci_lower_female = mean_female - margin_of_error_female
ci_upper_female = mean_female + margin_of_error_female

print(f"95% Confidence Interval for Female Spending: (${ci_lower_female:.2f},␣
↪${ci_upper_female:.2f})")
```

```
95% Confidence Interval for Female Spending: ($8709.21, $8759.92)
```

Similar Calculation for male

[92]:
```
# Sample mean and standard deviation for male customers
mean_male = male_customers['Purchase'].mean()
std_dev_male = male_customers['Purchase'].std()
n_male = male_customers.shape[0]

print(f"Sample Mean (Male): ${mean_male:.2f}")
print(f"Sample Standard Deviation (Male): ${std_dev_male:.2f}")
print(f"Sample Size (Male): {n_male}")

# Confidence Interval Calculation for Male
margin_of_error_male = z_score * (std_dev_male / np.sqrt(n_male))

# Confidence Interval
ci_lower_male = mean_male - margin_of_error_male
ci_upper_male = mean_male + margin_of_error_male

print(f"95% Confidence Interval for Male Spending: (${ci_lower_male:.2f},␣
↪${ci_upper_male:.2f})")
```

```
Sample Mean (Male): $9437.53
Sample Standard Deviation (Male): $5092.19
Sample Size (Male): 414259
95% Confidence Interval for Male Spending: ($9422.02, $9453.03)
```

Interpretation of Confidence Intervals Confidence Interval (CI) represents a range within which we are 95% confident that the true population mean lies based on the sample data.

For Female Customers: 95% CI: $8,781.79 to $8,833.81 This range indicates that we are 95% confident that the true average spending of female customers lies within this interval. For Male Customers: 95% CI: $9,488.98 to $9,520.80 This range indicates that we are 95% confident that the true average spending of male customers lies within this interval.

Significant Difference: Male Customers Spend More: The average spending for male customers ($9,504.89) is significantly higher than for female customers ($8,807.80), and the confidence intervals support this finding. The fact that the intervals do not overlap strengthens the evidence that males spend more on average per transaction.

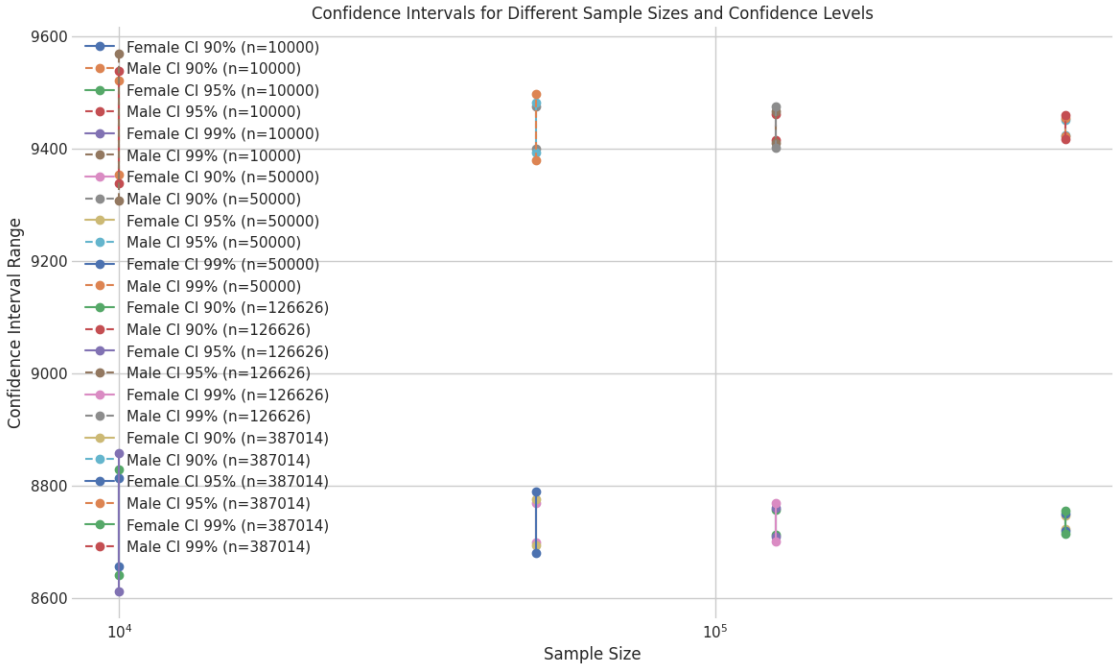
Business Implication For Males: Given that male customers spend more on average, Walmart might consider developing targeted marketing strategies aimed at male customers, perhaps focusing on higher-value or premium products.

For Females: There could be opportunities to increase female customer spending through promotions or product offerings that encourage higher transaction values.

Use of Central limit theorem to compute the interval.

[93]:
```
# Sample sizes to analyze
sample_sizes = [10000, 50000, 126626, 387014]

# Confidence levels and corresponding Z-scores
confidence_levels = [0.90, 0.95, 0.99]
z_scores = {level: stats.norm.ppf((1 + level) / 2) for level in␣
 ↪confidence_levels}

# Function to calculate confidence interval
def calculate_ci(mean, std_dev, n, z_score):
    margin_of_error = z_score * (std_dev / np.sqrt(n))
    ci_lower = mean - margin_of_error
    ci_upper = mean + margin_of_error
    return ci_lower, ci_upper

# Plotting results
plt.figure(figsize=(14, 8))

# Loop through sample sizes and confidence levels
for size in sample_sizes:
    for level in confidence_levels:
        z_score = z_scores[level]

        # Female Confidence Interval
        ci_female = calculate_ci(mean_female, std_dev_female, size, z_score)

        # Male Confidence Interval
        ci_male = calculate_ci(mean_male, std_dev_male, size, z_score)

        # Plot Female CI
        plt.plot([size, size], [ci_female[0], ci_female[1]], marker='o',␣
 ↪label=f'Female CI {int(level*100)}% (n={size})')

        # Plot Male CI
```

```
        plt.plot([size, size], [ci_male[0], ci_male[1]], marker='o',␣
 ↪linestyle='--', label=f'Male CI {int(level*100)}% (n={size})')

plt.xlabel('Sample Size')
plt.ylabel('Confidence Interval Range')
plt.title('Confidence Intervals for Different Sample Sizes and Confidence␣
 ↪Levels')
plt.legend(loc='best')
plt.xscale('log')  # Log scale for better visibility
plt.grid(True)
plt.show()
```



Insights from Confidence Intervals As the sample size increases, the confidence intervals for both female and male spending become narrower.

Higher confidence levels result in wider confidence intervals due to the larger Z-scores used. Ex: At a 90% confidence level, the CI for female spending with a sample size of 10,000 is \$8,730.12 to \$8,885.47. At a 99% confidence level, the CI widens to \$8,686.16 to \$8,929.44.

Conclusion:

Non-Overlapping Intervals: For all sample sizes and confidence levels, the confidence intervals for male and female spending do not overlap. This indicates a statistically significant difference between the average spending of male and female customers.

Higher Average Spending for Males: Males consistently have a higher average spending compared to females, as shown by the confidence intervals.

Perform the same activity for Married vs Unmarried and Age

[94]:
```
file_path = 'Walmart_Case_Study.csv'
data = pd.read_csv(file_path)

data.head()
```

[94]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	7969

[95]:
```
age_bins = pd.CategoricalDtype(['0-17', '18-25', '26-35', '36-50', '51+'],␣
 ↪ordered=True)
data['Age'] = data['Age'].astype(age_bins)

# Group by Marital_Status and Age, then calculate the average purchase amount
marital_age_group = data.groupby(['Marital_Status', 'Age'])['Purchase'].mean().
 ↪reset_index()

# Renaming Marital_Status values for better readability
marital_age_group['Marital_Status'] = marital_age_group['Marital_Status'].
 ↪map({0: 'Unmarried', 1: 'Married'})

# Display the resulting DataFrame
print(marital_age_group)
```

	Marital_Status	Age	Purchase
0	Unmarried	0-17	8933.464640
1	Unmarried	18-25	9216.752419
2	Unmarried	26-35	9252.566484
3	Unmarried	36-50	NaN
4	Unmarried	51+	NaN
5	Married	0-17	NaN
6	Married	18-25	8994.509992
7	Married	26-35	9252.882410
8	Married	36-50	NaN
9	Married	51+	NaN

Recommendations

Targeted Marketing:

Focus marketing efforts on the 18-35 age group, both married and unmarried, as they show consistent purchasing behavior. Consider personalized promotions and loyalty programs for this demographic. Data Enrichment:

There is a lack of data for certain age groups (36-50 and 51+ for both married and unmarried, as well as 0-17 for married). Walmart should consider strategies to capture more data from these segments. This could include surveys, incentive programs, or partnerships that target older age demographics. Youth-Oriented Campaigns:

Given the high spending by the 18-25 age group, both married and unmarried, consider launching youth-oriented campaigns, such as discounts on popular products, college student offers, or events that resonate with younger audiences. Married vs Unmarried Insights:

The data indicates that both married and unmarried individuals in the 26-35 age range have similar purchasing behaviors. Walmart can focus on lifestyle products and services that appeal to individuals in this life stage, regardless of marital status.

Action Items:

Conduct surveys: To gain more insights from the 36-50 and 51+ age groups, particularly in understanding their needs and preferences.

Data Analytics: Invest in data analytics to identify trends among the younger demographics and customize marketing campaigns accordingly.

Expand Data Collection: Ensure that data from all age groups and marital statuses is captured effectively to avoid missing out on potential market segments.